

CS F364 : Design & Analysis of Algorithms

Project Report

Efficient Algorithms for Densest Subgraph Discovery and Extracting
Densest Subgraphs Without Flow Computations

Group 26



Group Members

Kriti Saluja	(2022B5A70698H)
M Adithya Krishna	(2023A7PS0144H)
Mriganka Thakur	(2022B5A71309G)
Vishal Goel	(2023A7PS1108H)
Veer Gandhi	(2023A7PS1234H)

1 Introduction:

Densest subgraph discovery (DSD) is a fundamental problem in graph mining. It has been studied for decades, and is widely used in various areas, including network science, biological analysis, and graph databases. Given a graph G , DSD aims to find a subgraph D of G with the highest density (e.g., the number of edges over the number of vertices in D).

The densest subgraph problem (DSP) that asks to find a subgraph with maximum average degree forms a basic primitive in dense subgraph discovery with applications ranging from community detection to unsupervised discovery of biological network modules.

The triangle densest subgraph problem aims to find a subset of vertices that maximizes the number of triangles per node, thereby capturing highly cohesive and mutually connected groups.

Formally, given an undirected graph $G=(V,E)$, the objective is to determine a subset

$$S \subseteq V$$

that maximizes:

$$\frac{|T(S)|}{|S|}$$

where $T(S)$ denotes the number of triangles induced by S . This measure reflects the presence of strong transitive relationships and is particularly useful in applications such as community detection, social network analysis, and biological network modeling.

In this work, we implement and analyze algorithms designed to efficiently identify triangle-dense subgraphs, focusing on their effectiveness and scalability across different datasets through testing.

2 Algorithms Implemented:

1. **Charikar’s Greedy Peeling:** This algorithm iteratively removes the vertex with the minimum degree and tracks the subgraph with the highest edge density encountered during the process. It is simple, efficient, and provides a approximation for the densest subgraph problem.
2. **Greedy++:** Greedy++ extends Charikar’s approach by performing multiple peeling passes and introducing a load-based weighting mechanism. In each iteration, node selection is based on a combination of current degree and accumulated load, leading to improved practical performance over the standard greedy method.
3. **PeelApp (Triangle Peeling Approximation):** PeelApp is a triangle-based extension of the greedy peeling strategy. Instead of using vertex degree, it considers the number of triangles each vertex participates in (triangle-degree). Vertices with the smallest triangle-degree are removed iteratively, and the best triangle density subgraph is tracked. This method provides an approximate solution for the triangle densest subgraph problem.
4. **Triangle-Core Decomposition:** This algorithm generalizes the concept of k-core decomposition to triangles. Each vertex is assigned a core number based on its participation in triangles, and vertices are iteratively removed based on triangle-degree. The resulting decomposition is used to identify structurally dense regions and to prune the search space for more expensive algorithms.

5. **CoreExact (Flow-Based Exact Algorithm):** CoreExact computes the exact triangle densest subgraph using a combination of binary search and max-flow techniques. For a given density threshold, a flow network is constructed and solved using Dinic's algorithm. The solution is refined iteratively through binary search to obtain the optimal density. To improve efficiency, the algorithm operates on core-reduced subgraphs and uses approximate results as lower bounds to prune the search space.

3 Individual Contributions

- **Kriti Saluja (ID: 2022B5A70698H):** Implemented the 5 algorithms mentioned above based on the reference paper.
- **M Adithya Krishna (ID: 2023A7PS0144H):** Worked on executing the implementation on the provided datasets and verifying correctness of outputs.
- **Mriganka Thakur (ID: 2022B5A71309H):** Assisted in testing and improving the performance of the implementation through optimizations.
- **Vishal Goel (ID: 2023A7PS1108H):** Contributed to writing and structuring the project report.
- **Veer Gandhi (ID: 2023A7PS1234H):** Assisted in documentation and preparation of the final report.

4 System Configuration

All experiments were conducted on a Linux-based system with the following configuration:

- Operating System: Linux (Pop-OS)
- Processor: 13th Gen Intel(R) Core(TM) i5-13420H
- RAM: 16 GB
- CPU Utilization: 99
- Compiler: g++ with optimization flags `-O3 -march=native -std=c++17`

4.0.1 Combined (Email-Enron)

Execution Time:

- Algorithm Time: 29.115 seconds
- CoreExact algorithm time: 28.362 seconds

Memory Usage:

- Approximate Memory (Code Estimate): 161.0 MB
- Resident Set Size after algorithms(RSS): 164 MB
- Resident Set Size before algorithms(RSS): 3 MB

4.0.2 Combined (Wiki-Vote)

Execution Time:

- Algorithm Time: 20.658 seconds

- CoreExact algorithm time: 20.293 seconds

Memory Usage:

- Approximate Memory (Code Estimate): 117.0 MB
- Resident Set Size after algorithms(RSS): 120 MB
- Resident Set Size before algorithms(RSS): 3 MB

4.0.3 Combined (as-skitter)

Execution Time:

- Algorithm Time: 10787.838 seconds
- CoreExact algorithm time: 9914.723 seconds

Memory Usage:

- Approximate Memory (Code Estimate): 5 GB
- Resident Set Size after algorithms(RSS): 5 GB
- Resident Set Size before algorithms(RSS): 3 MB

5 Output of the Program

5.1 Dataset 1: Email-Enron

Graph Statistics:

- Number of Nodes: 36,692
- Number of Edges: 183831

edge-density:

```
[Charikar greedy (edge-density, 1/2-approx)]
  Edge-density      : 37.344144
  Subgraph size     : 555 vertices
[Greedy++ (edge-density, 5 passes)]
  Edge-density      : 37.344144
  Subgraph size     : 555 vertices
[PeelApp (triangle-density peeling, 1/3-approx)]
  Triangle-density  : 404.451948
  Subgraph size     : 385 vertices
[CoreExact (triangle-density, Dinic flow, exact)]
  Triangle-density  : 404.456186
  Subgraph size     : 388 vertices
```

5.2 Dataset 2: Wiki-Vote

Graph Statistics:

- Number of Nodes: 7,115
- Number of Edges: 100762

edge-density:

```

[Charikar greedy (edge-density, 1/2-approx)]
  Edge-density      : 46.275414
  Subgraph size     : 846 vertices
[Greedy++ (edge-density, 5 passes)]
  Edge-density      : 46.278846
  Subgraph size     : 832 vertices
[PeelApp (triangle-density peeling, 1/3-approx)]
  Triangle-density  : 504.329741
  Subgraph size     : 464 vertices
[CoreExact (triangle-density, Dinic flow, exact)]
  Triangle-density  : 504.334764
  Subgraph size     : 466 vertices

```

5.3 Dataset 3: as-skitter

Graph Statistics:

- Number of Nodes: 1696415
- Number of Edges: 11095298

edge-density:

```

[Charikar greedy (edge-density, 1/2-approx)]
  Edge-density      : 89.117096
  Subgraph size     : 427 vertices
[Greedy++ (edge-density, 5 passes)]
  Edge-density      : 89.400862
  Subgraph size     : 464 vertices
[PeelApp (triangle-density peeling, 1/3-approx)]
  Triangle-density  : 3366.622328
  Subgraph size     : 421 vertices
[CoreExact (triangle-density, Dinic flow, exact)]
  Triangle-density  : 3366.622328
  Subgraph size     : 0 vertices
(WARNING: timed out during binary search)

```

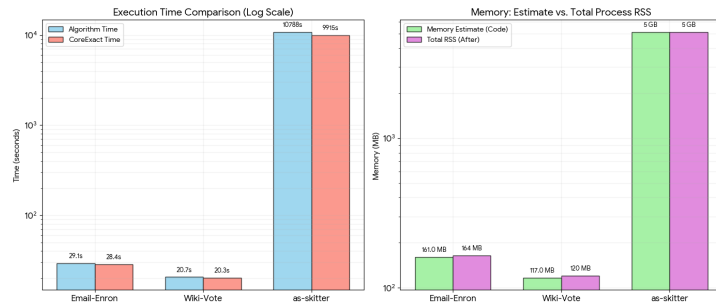


Figure 1: Execution Time and Memory usage Comparison between Wiki-Vote, as-skitter and Email-Enron datasets